



Persistence - Relational DBs

مقدمه

در این پروژه باید تمام داده‌های موردنیاز برنامه را که در فازهای قبلی در حافظه برنامه ذخیره و استفاده می‌شدند، در یک پایگاه داده با حافظه ماندگار نگهداری کنید. ابتدا لازم است شمای پایگاه داده را طراحی کنید و سپس، آن‌ها را در پایگاه داده ذخیره کرده و هنگام نیاز بازیابی کنید. بخش فرانت‌اند پروژه در این فاز تغییری نمی‌کند.

در این پروژه از پایگاه داده‌های رابطه‌ای استفاده خواهد شد. در پایگاه داده رابطه‌ای، داده‌ها در جدول‌هایی با تعداد مشخصی از ویژگی‌ها و نوع داده معین ذخیره می‌شوند. شباهت زیادی میان ویژگی‌های موجودیت‌های پایگاه داده و ویژگی‌های اشیای دامنه برنامه در طراحی شیء‌گرا وجود دارد و همین امر موجب می‌شود ارتباط و تبدیل میان این دو آسان‌تر شود. برای انجام این تبدیل لازم است از ORM که در درس با آن آشنا شده‌اید استفاده کنید.

پایگاه داده

JDBC

JDBC یا Java Database Connectivity، یک رابط استاندارد برای برقراری ارتباط میان برنامه‌های جاوا و پایگاه داده است. این API به برنامه‌نویس اجازه می‌دهد که پس از اتصال برنامه به پایگاه داده، با استفاده از دستورات SQL، به صورت مستقیم روی پایگاه داده کوئری اجرا کند و نتایج آن را دریافت نماید.

در این روش:

- برنامه‌نویس باید به صورت دستی اتصال به پایگاه داده را مدیریت کند.
- کوئری‌های SQL باید مستقیماً به صورت رشته درکد نوشته شوند.
- نتایج کوئری‌ها به صورت ResultSet بازگردانده می‌شوند و باید با پیمایش Cursor آن، سطرهای جدول به صورت دستی به اشیای دامنه برنامه تبدیل شوند.

انجام این فرایند به صورت دستی برای همه اشیای برنامه، منجر به تولید حجم زیادی از کدهای تکراری می شود که نگهداری آن ها دشوار است و احتمال بروز خطا در آن ها نیز بالا خواهد بود. بنابراین، با وجود آنکه JDBC امکان استفاده از تمامی قابلیت های پایگاه داده را فراهم می کند، ابزارهای دیگری برای ارائه سطحی انتزاعی تر توسعه یافته اند تا این فرایند را ساده تر کنند.

فریم ورک های ORM یا Object Relational Mapper در جاوا، با استفاده از Annotation ها و Metadata ها، نداشت میان جدول های پایگاه داده و کلاس های دامنه را ساده تر و خودکار می کنند.

JPA

JPA یا Java Persistence API، یک استاندارد رسمی برای پیاده سازی ORM در زبان جاوا است. این استاندارد به برنامه نویس اجازه می دهد که کلاس های جاوا را به جدول های پایگاه داده نگاشت کند.

JPA با استفاده از Annotation هایی مانند:

@Entity, @Table, @Id, @GeneratedValue, @Basic, @Column, @Enumerated,
@Embeddable, @Embedded, @OneToOne, @OneToMany, @ManyToOne, @ManyToMany,
@JoinColumn, @OnDelete, @Inheritance @DiscriminatorColumn @DiscriminatorValue

مشخص می کند که هر کلاس جاوا چگونه باید به جدول های پایگاه داده نگاشت شود. شما در کلاس با این API آشنا شدید و مشاهده کردید که عملیات CRUD، یعنی Create, Read, Update و Delete، همچنین مدیریت تراکنش ها نیز از طریق EntityManager انجام می شوند.

در صورتی که در یک کلاس، یک فهرست از نوع OneToMany ایجاد شود و در کلاس دیگر نیز فیلد متناظر آن به صورت ManyToOne وجود داشته باشد، با استفاده از mappedBy مشخص می کنیم که کلید خارجی در کدام جدول قرار می گیرد.

توجه داشته باشید که JPA صرفاً یک استاندارد است که مشخصات API را مستند می کند و خود یک پیاده سازی مستقل نیست. برای آشنایی بیشتر با JPA و Annotation های یاد شده، مطالب این [لینک](#) را مطالعه کنید.

Hibernate

Hibernate یک فریم‌ورک ORM برای زبان جاوا است. این ابزار در واقع یکی از پیاده‌سازی‌های مشهور استاندارد JPA به شمار می‌آید که امکانات پیشرفته‌تری را نیز در اختیار توسعه‌دهنده قرار می‌دهد. استفاده از این ابزار در پروژه شما الزامی است.

Hibernate پس از تنظیم صحیح موجودیت‌ها با Annotation‌های مربوط به JPA، به صورت خودکار بر اساس کلاس‌های برنامه، جدول‌های متناظر را در پایگاه‌داده ایجاد می‌کند، داده‌ها را نگاشت می‌کند و کوئری‌های موردنیاز را با زبان اختصاصی خود، یعنی HQL، تولید می‌نماید. این چارچوب در مدیریت ارتباط میان موجودیت‌ها، کش کردن داده‌ها، بارگذاری تنبل و بهینه‌سازی جست‌وجوها کاربرد بسیار زیادی دارد. در نهایت، Hibernate برای برقراری ارتباط با پایگاه‌داده از JDBC استفاده می‌کند، اما این جزئیات را از دید برنامه‌نویس پنهان می‌سازد.

Spring Data & Repository

Repository در واقع یک لایه سطح بالاتر است که توسط فریم‌ورک Spring Data ارائه می‌شود تا کار با JPA و Hibernate را ساده‌تر کند. در این روش، برنامه‌نویس برای هر Entity تنها یک Interface تعریف می‌کند که از یک یا چند مورد از Repository‌های Spring Data مانند CrudRepository، ListCrudRepository، PagingAndSortingRepository و موارد مشابه ارث‌بری می‌کند. با این کار، متدهای آماده برای ذخیره، ویرایش، حذف و جست‌وجوی موجودیت‌ها در اختیار او قرار می‌گیرد.

مزیت اصلی Repository این است که دیگر نیازی به نوشتن پیاده‌سازی متدها وجود ندارد، زیرا فریم‌ورک Spring این کار را به صورت خودکار انجام می‌دهد. یکی از قابلیت‌های مهم آن، Query Methods است؛ به این معنا که می‌توان متدهای جست‌وجو بر اساس نام، تاریخ، شناسه و موارد دیگر را صرفاً با نام‌گذاری صحیح متد تعریف کرد. برای مثال، با ایجاد متدی به نام findByIdAndDate، کد مربوط به اجرای این جست‌وجو به طور خودکار تولید می‌شود.

در Repository این امکان وجود دارد که در صورت نیاز، با استفاده از @Query یک پرس‌وجوی مستقیم روی پایگاه‌داده اجرا شود. همچنین برای انجام جست‌وجوهای پیچیده‌تر و اعمال شرط‌های متعدد در لایه پایگاه‌داده، می‌توان از API Criteria در JPA و به طور مشخص از JpaSpecificationExecutor استفاده کرد که امکان توسعه‌پذیری بیشتری برای فیلترهای پیشرفته فراهم می‌کند.

این لایه باعث افزایش سرعت توسعه و بهبود خوانایی کد می‌شود. برای آشنایی بیشتر می‌توانید بخش Spring Data JPA را در [لینک](#) مربوط مطالعه کنید. برای افزودن قابلیت‌های ذکرشده به‌همراه Hibernate، معمولاً اضافه کردن spring-boot-starter-data-jpa و کانکتور MySQL به dependencyهای Maven کافی است.

توجه داشته باشید که هنگام import کردن پکیج‌های مربوط به servlet و persistence (که بخشی از Java EE / Jakarta EE هستند)، باید از مسیر jakarta و نه javax استفاده کنید.

نکات پایانی

- این پروژه به صورت انفرادی است.
- برای تحویل پروژه کد های خود را zip و در صفحه درس بارگذاری کنید.
- انتخاب پایگاه‌داده رابطه‌ای مناسب برای پروژه یکی از مضایف شما است، برای انتخاب خود دلیل داشته باشید.
- در صورت عدم استفاده از Hibernate بخش اعظمی از نمره شما کسر خواهد شد.
- دقت کنید تعریف کلید اصلی و کلید خارجی مناسب برای جدول‌هایان ضروری است.
- برای ذخیره‌سازی اطلاعات مختلف از Data Type های مناسب در شمای رابطه استفاده کنید.
- توصیه می‌شود از Repository های Spring استفاده کنید.
- هدف این تمرین یادگیری و دور کردن ذهن شما از شرایط فعلی است. لطفا تمرین را خودتان انجام دهید.